THE MARKDOWN TO HTML PIPELINE

(you are now familiar with these steps)

knit text and output together

```
---
title: "MARKDOWN DEMO"
output: pdf_document
---


# Here is the heading

Here is some text.

## A sub-heading

Perhaps you want a list:

- Item 1
- Item 2

And here is some code:

```{r}

plot( cars, col="steel blue", cex=1.5, pch=19 )

```
```

(header)

(text)

(code)

MARKDOWN DEMO

Here is the heading

Here is some text.

A sub-heading

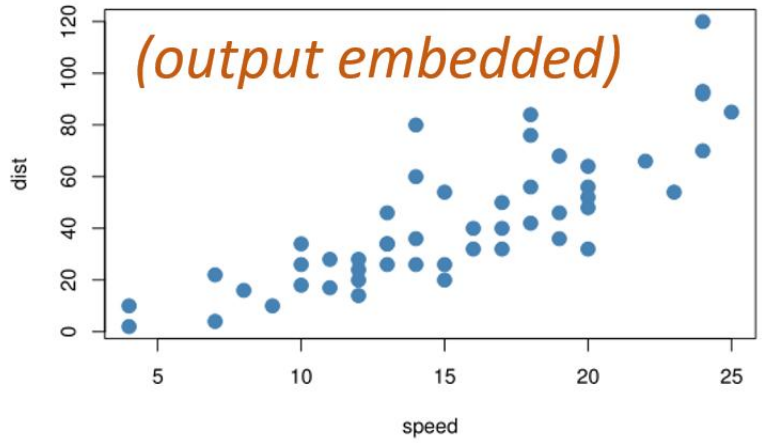
Perhaps you want a list:

- Item 1
- Item 2

And here is some code:

```
plot( cars, col="steel blue", cex=1.5, pch=19 )
```

(rendered PDF)



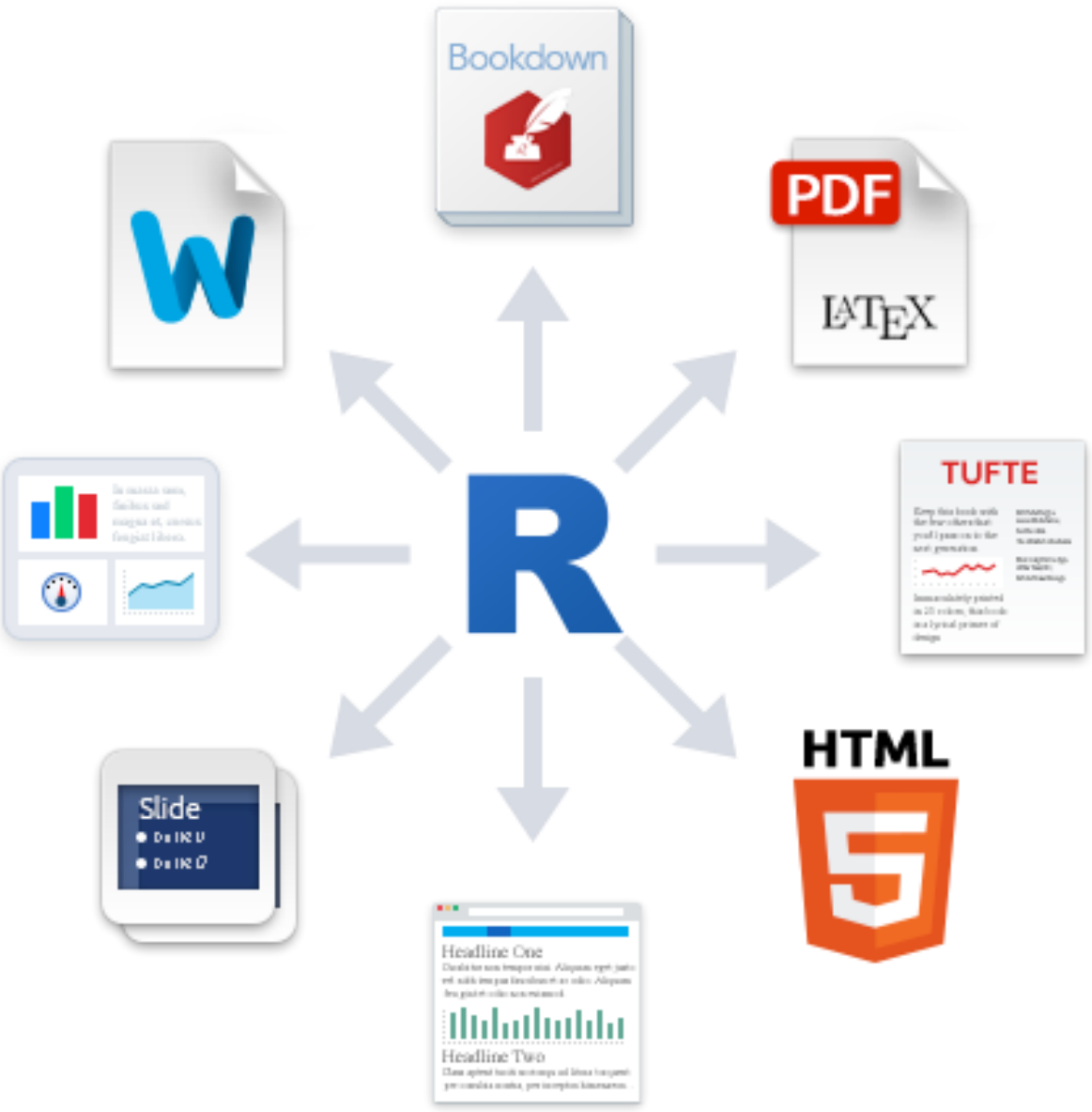
“Knit” your R markdown document in RStudio

`pandoc` translates the markdown file to whichever file type you desire: html, doc, pdf, website, dashboard

The `knitr` package translates the R markdown file to a regular markdown file

Anatomy of the R Markdown workflow when you knit a file in RStudio

One reason RMD docs are powerful is Pandoc can render content into a variety of content types including HTML docs, word docs, slide decks, dashboards, and PDFs (to name a few)



MARKDOWN TO HTML

MARKDOWN

MD+R
=RMD

RStudio

KNITR → **PANDOC** → **HTML**

When knitting, markdown text is converted to the proper text elements (titles, paragraphs, lists, tables, links, etc.) and each element is nicely formatted to create a readable and professional HTML document.

GITHUB PAGES

MARKDOWN TO HTML

MARKDOWN → **JEKYLL** → **HTML**

GitHub

- GH Pages
- YAML settings
- Liquid Tags

- Tags
- Style (CSS)
- Page elements (head, header, body, footer)

MD+R
=RMD

RStudio

KNITR → PANDOC

Now, instead of knitting RMD docs, we will be converting Markdown files to HTML using GitHub pages. Jekyll is equivalent to the KnitR/Pandoc framework used to render RMD files into HTML.

MARKDOWN TO HTML

MARKDOWN → **JEKYLL** → **HTML**

GitHub

- GH Pages
- YAML settings
- Liquid Tags

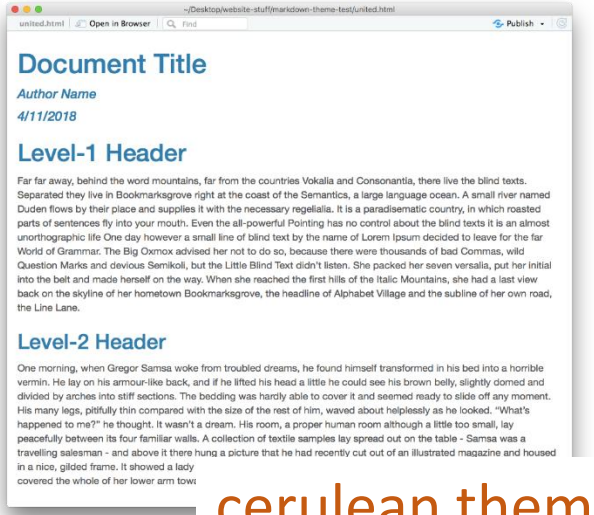
- Tags
- Style (CSS)
- Page elements (head, header, body, footer)

In order to start building page templates, you will need to understand a few simple rules about how GitHub pages work, a few simple rules about how HTML files are organized, and some basic CSS.

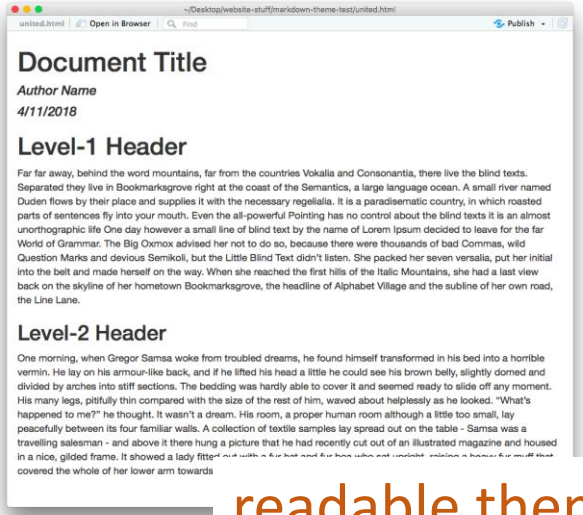
CSS: CASCADING STYLE SHEETS

Why use CSS?

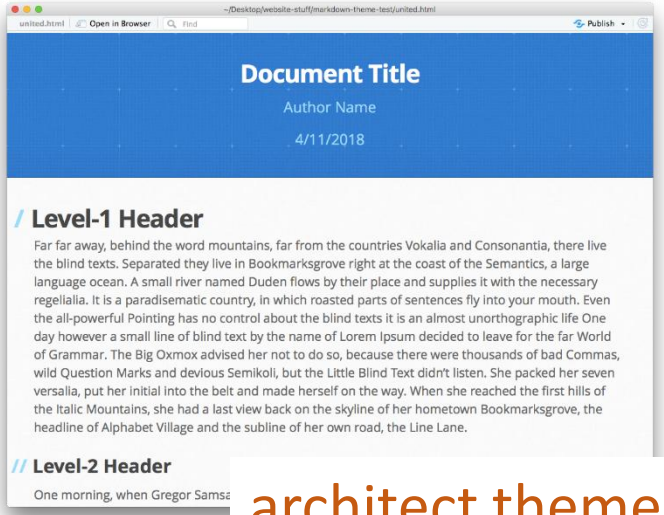
- Separates the task of creating content from packaging content.



cerulean theme



readable theme



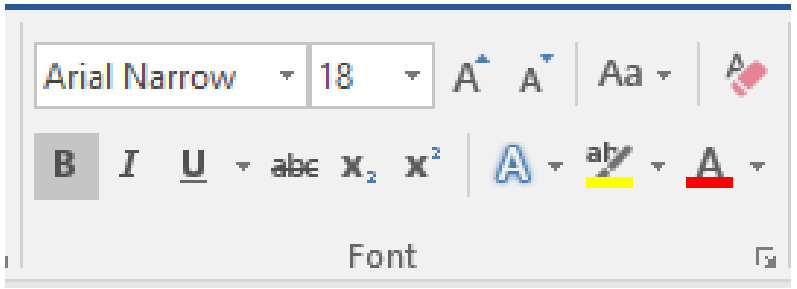
architect theme

```
---
output:
  html_document:
    theme: readable
---
```

You've been using this in RMD documents already !

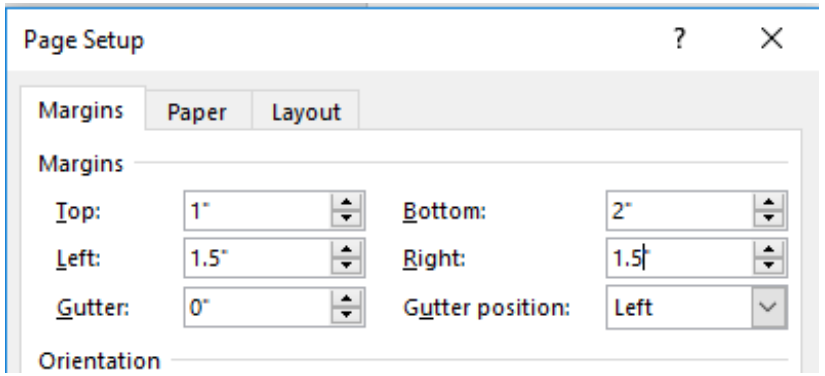
What is a STYLE?

Graphical User Interface Version of Style



(text style)

```
h1{
   font-type: arial narrow;
   font-decoration: bold;
   font-size: 18pt;
   color: red;
}
```



(page style)

```
body {
   margin-top: 1;
   margin-left: 1.5;
   margin-right: 1.5;
   margin-bottom: 2;
}
```

h2 = ## header 2 style

the CSS defines the aesthetics of each type of content on the page

```
h2 {
  color: #333333;                    /* Changes text color to a dark gray */
  font-size: 1.5rem;                 /* Sets the font size */
  font-family: Arial, sans-serif;    /* Specifies the font */
  margin-top: 1em;                   /* Adds space above the heading */
  margin-bottom: 0.5em;              /* Adds space below the heading */
  border-bottom: 2px solid #333333;  /* Adds a border to the bottom */
}
```

HTML PAGE BASICS

HTML PAGE

```
<html>

  <head>...</head>

  <body>

    <header>...</header>

    <p> ... </p>

    <footer>...</footer>

  </body>

</html>
```

First you must understand the bare minimum elements you need on an HTML page.

- **HEAD:** load html packages, fonts, and other resources needed for the page
- **BODY:** start of page content
- **HEADER:** top of a page that often contains the page title and navigation bars
- **MAIN or ARTICLE:** section that contains the actual text
- **FOOTER:** elements at the bottom of a page

Note that elements are hierarchical – the body section contains the header, main, and footer sections.

PAGE ELEMENTS

```
<html>

  <head>...</head>

  <body>

    <header>...</header>

    <p> . . . </p>

    <footer>...</footer>

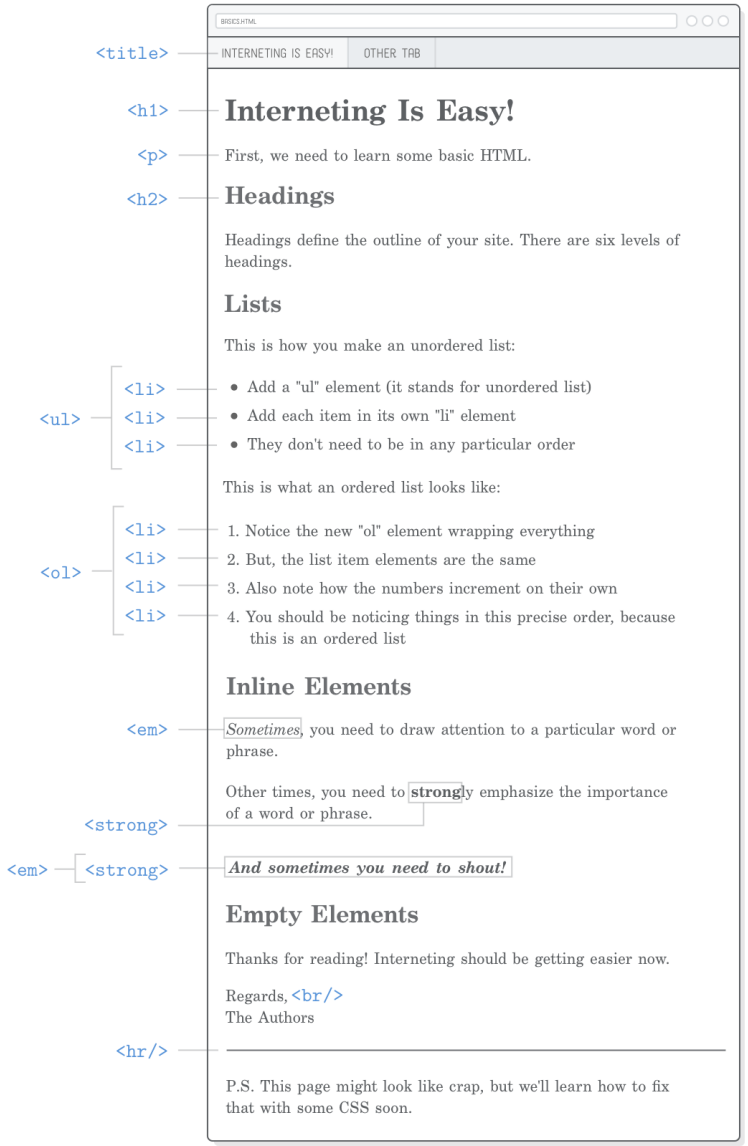
  </body>

</html>
```

CONTENT ELEMENTS

tags that organize content into sections

tags that define content types and formatting



titles

text

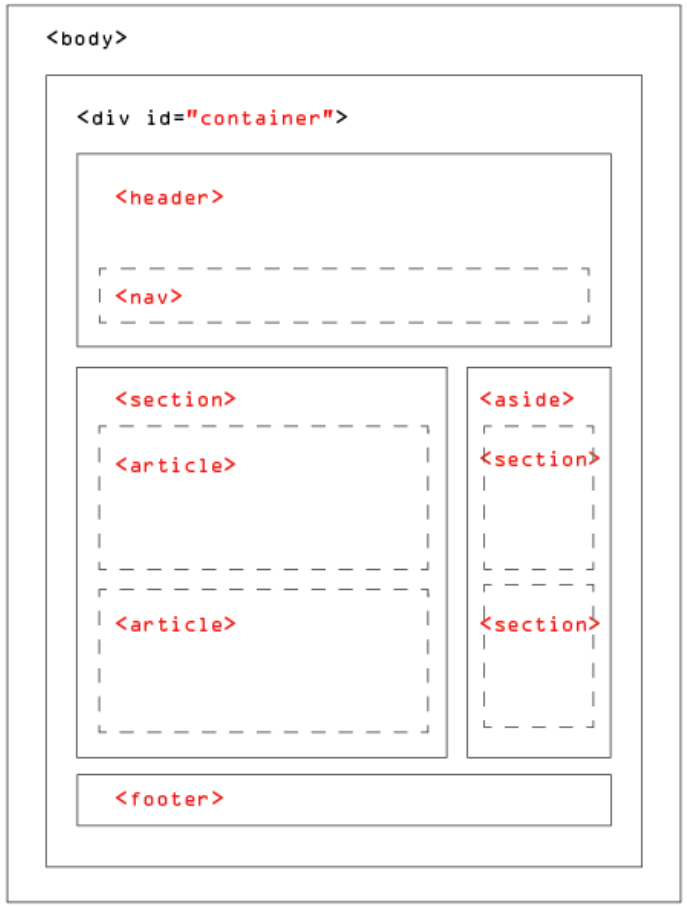
lists

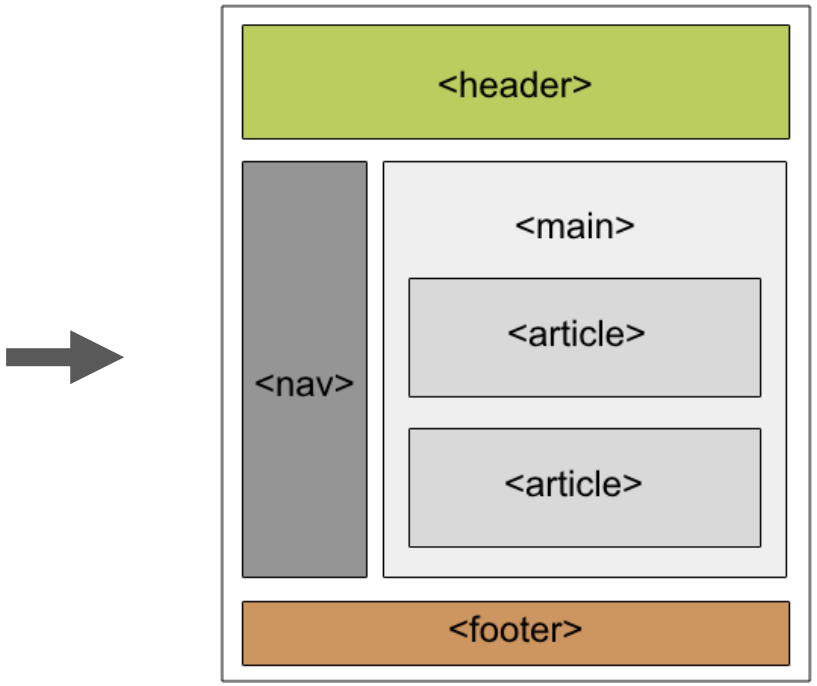
tables

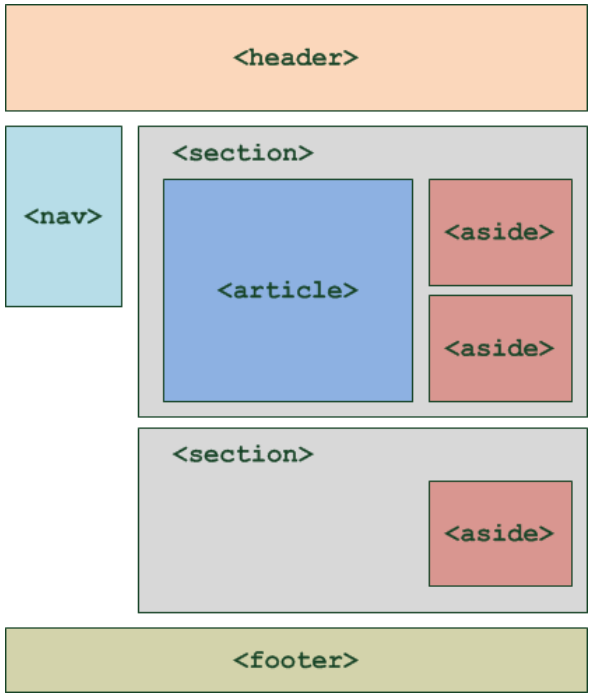
bold

italics

As you get more comfortable with HTML you can create more sophisticated layouts and add new page elements. Page layout is an advanced CSS skill and also an easy task to contract out to someone that specializes in website design.







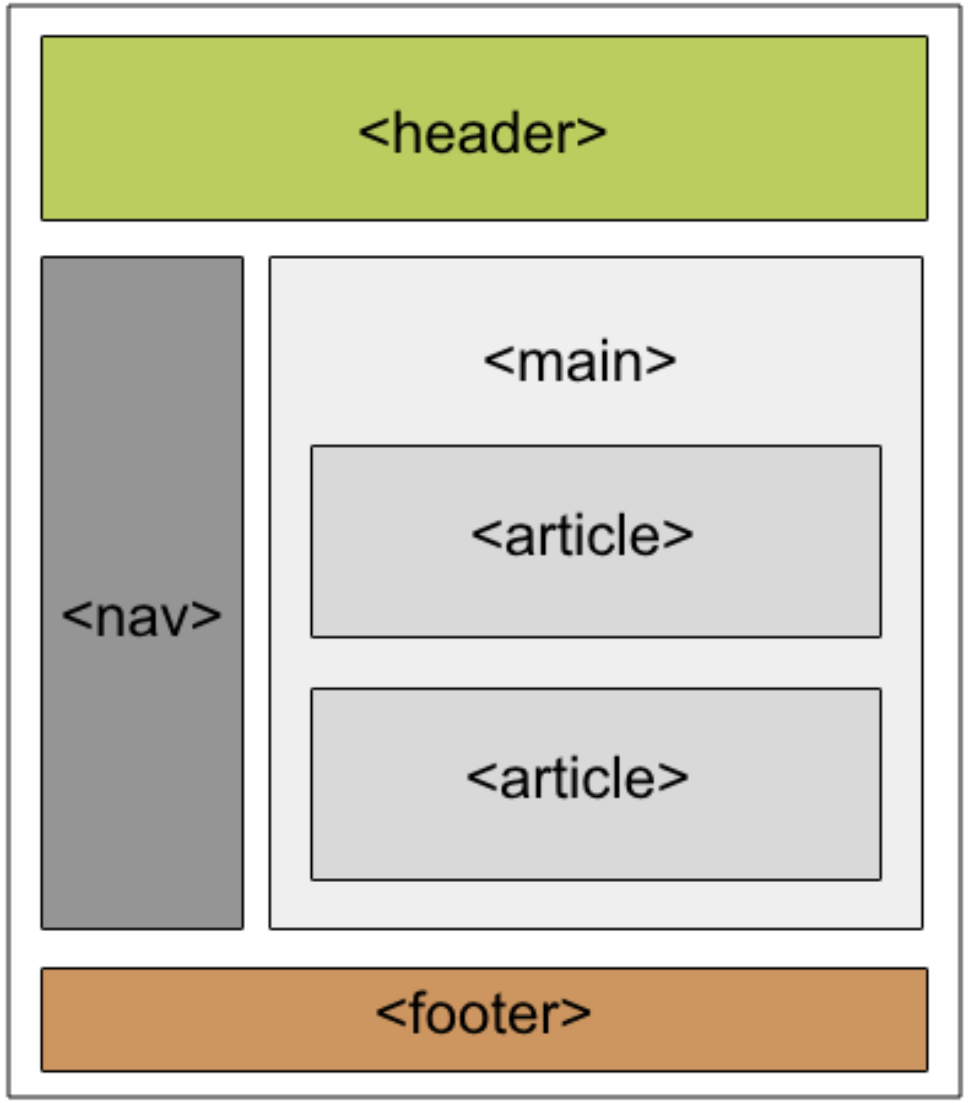
THE JEKYLL FRAMEWORK

/includes folder

header.html

nav.html

footer.html



Generic page elements that are reused throughout the site are defined and saved as separate files. They are imported wherever needed. These are the templated elements.

Page content comes from the text contained in the markdown file itself and is specific to the page. Content is NOT templated.

LIQUID / JEKYLL → HTML

```
<html>

  {% include head.html %}

  <body>

      {% include header.html %}

      {{ content }}

      {% include footer.html %}

  </body>

</html>
```

```
<html>

  <head>…</head>

  <body>

    <header>…</header>

    <p> ... </p>

    <footer>…</footer>

  </body>

</html>
```

LIQUID / JEKYLL ← HTML

The {% include %} liquid tag is the way that Jekyll combines page elements by replacing the tag with the actual HTML code contained within the referenced file

```
<html>
<body>

{% include header.html %}

</body>
</html>
```

header.html file content:

```
<header>
<div class="navbar">

 <a id="site-title" href="index"> {{ site.title }} </a>

 <a href="index">  Home          </a>
 <a href="page1">  Layout Example  </a>
 <a href="page2">  Table Example   </a>
 <a href="about">  About         </a>

</div>
</header>
```

LIQUID TAGS

Generic Variables

- Liquid tags allow you to reference info in your YAML header directly in your page.

```
---
yellowdig:
  post-01: 'Monday, Aug 24th’
  post-02: 'Monday, Aug 31st'
---
```

YAML header data

```
## Practice Problem Warmup
```

Liquid tag referencing YAML field

```
**Post on {{page.yellowdig.post-01}}**
```

```
Note that these are non-obvious bugs that can EASILY work there way into your code. Once you see
the problem, it will seem obvious. But until you see it the code often looks fine and it's
unclear why it is not working as expected.
```

Generic Variables

(Rendered HTML Page)

Practice Problem Warmup

Liquid tag replaced with value in YAML table

Post on Monday, Aug 24th

Note that these are non-obvious bugs that can EASILY work there way into your code. Once you see the problem, it will seem obvious. But until you see it the code often looks fine and it's unclear why it is not working as expected.

These practice problems are largely an opportunity to review content from CPP 526 and develop a sensitivity for the importance of writing robust and sustainable code, and testing your code for unexpected behaviors.

Many of these questions are similar to riddles or brain teasers that help sharpen your critical thinking skills when working with code. Work through them on your own, then discuss your ideas on YellowDig with classmates.